week04

1. 객체지향 프로그래밍의 필요성

OOP(Object Oriented Programming)

- 모든 코드를 main() 함수 안에 작성하면 코드 정리가 안되고 구조화가 되지 않는다

→ 변수와 메서드를 특정 클래스에 종속되게 코딩할 수 있다

인스턴스==객체

클래스를 이용해서 객체를 선언하면 해당 객체를 클래스의 인스턴스라고 한다

```
void main(){
  Idol blackPink = Idol();
  blackPink.sayName();
}

class Idol{
  String name='블랙핑크'; //Idol 클래스에 종속된 변수

  void sayName(){
    print('저는 ${this.name}입니다.');
  }
}
```

2. 생성자

외부에서 받은 값을 넣을 수 있다

```
class Idol{
  final String name;
  final List<String> members;

  Idol(String name, List<String> members)
    :this.name = name,
    this.members = members;
```

```
  void sayHello(){
    print('안녕하세요 ${this.name}입니다.');
  }
  void introduce(){
    print("저희 멤버는 ${this.members}입니다.");
  }
}

void main(){
  Idol blackPink = Idol('블랙핑크', ['지수', '제니', '리사',
'로제']);
  blackPink.sayHello();
  blackPink.introduce();
}
```

네임드 생성자

```
class Idol{
    final String name;
    final int membersCount;

    Idol(String name, int membersCount)
        :this.name=name,
        this.membersCount=membersCount;
        //그냥 생성자

    Idol.fromMap(Map<String, dynamic> map)
        :this.name=map['name'],
        this.membersCount=map['membersCount'];

    void sayName(){
        print('저는 ${this.name}입니다. ${this.membersCount}
명입니다.');
    }
}
```

```
void main(){
    Idol blackPink = Idol('블랙핑크', 4);
    blackPink.sayName();

    Idol bts = Idol.fromMap({
        'name' : 'BTS',
        'membersCount' : 7,
        });
        bts.sayName();
}
```

💡 **네임드 생성자를 쓰는 이유**

1. 무슨 용도로 만드는 건지 이름을 붙일 수 있다
2. 외부 데이터(API, DB)를 다룰 때 필수이다.

→ 서버는 JSON 형태로 보내주는데 이때 사용하면 편하다

3. 생성자가 여러개 필요할 때 사용

프라이빗 변수

- 변수를 언더바(_)로 시작해 선언하면 같은 파일 내에서만 접근 가능한 변수라는 뜻이다

```
class Idol{
    String _name;

    Idol(this.name);
}

void main(){
    Idol blackPink = Idol('블랙핑크');
    print(blackPink._name);
}
```

getter/setter

- 어떤 값이 노출되고 어떤 형태로 노출될 지 그리고 어떤 변수를 변경 가능하게 할 지 유연하게 정할 수 있다
- 최근에 객체지향 프로그래밍을 할 때 변수의 값에 불변성(인스턴스화 후 변경할 수 없는)을 특성으로 사용하기 때문에 세터는 거의 사용하지 않지만 getter는 종종 사용하고 이런 게 있따 정도로 알고 넘어가면 된다.